

Correctness Proof of the Floyd–Warshall All-Pairs Shortest Path Algorithm

Setup and Definitions

Let $G = (V, E, w)$ be a weighted directed graph where $w : E \rightarrow \mathbb{R}$ assigns arbitrary real-valued weights to edges. We assume G contains no negative-weight cycle. Let $n = |V|$ and identify the vertices with $\{1, 2, \dots, n\}$.

Definition 1. For a path $p = \langle v_0, v_1, \dots, v_k \rangle$, its *weight* is $w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$. The *shortest-path distance* from i to j is

$$\delta(i, j) = \begin{cases} \min\{w(p) : p \text{ is a path from } i \text{ to } j\} & \text{if such a path exists,} \\ \infty & \text{otherwise.} \end{cases}$$

Definition 2. For $k \in \{0, 1, \dots, n\}$, define

$d^{(k)}[i][j]$ = weight of a shortest path from i to j whose intermediate vertices all belong to $\{1, \dots, k\}$.

An *intermediate vertex* of a path $\langle v_0, v_1, \dots, v_\ell \rangle$ is any vertex v_r with $0 < r < \ell$ (i.e., neither the source nor the destination).

The key observation is that $d^{(n)}[i][j] = \delta(i, j)$ for all i, j , since when $k = n$ no restriction is placed on intermediate vertices.

Algorithm

After the k -th iteration of the outer loop, the matrix d stores the values $d^{(k)}[i][j]$ for all pairs (i, j) .

Correctness

Theorem 1 (Correctness). *When Floyd–Warshall terminates, $d[i][j] = \delta(i, j)$ for every pair $i, j \in V$.*

The proof rests on the following loop invariant.

Algorithm 1 Floyd-Warshall(G)

1: **Initialize:** for all $i, j \in V$:

$$d[i][j] \leftarrow \begin{cases} 0 & \text{if } i = j, \\ w(i, j) & \text{if } (i, j) \in E, \\ \infty & \text{otherwise.} \end{cases}$$

2: **for** $k = 1$ **to** n **do**

▷ allow vertex k as a new intermediate

3: **for** $i = 1$ **to** n **do**

4: **for** $j = 1$ **to** n **do**

5: **if** $d[i][k] + d[k][j] < d[i][j]$ **then**

6: $d[i][j] \leftarrow d[i][k] + d[k][j]$

▷ route through k is shorter

7: **end if**

8: **end for**

9: **end for**

10: **end for**

11: **return** d

Lemma 1 (Loop Invariant). *After the k -th iteration of the outer loop ($k = 0, 1, \dots, n$), for every pair $i, j \in V$,*

$$d[i][j] = d^{(k)}[i][j].$$

Proof. We proceed by strong induction on k .

Base case ($k = 0$). Before the loop begins, the initialisation sets $d[i][i] = 0$ and, for $i \neq j$, $d[i][j] = w(i, j)$ if the direct edge exists, else ∞ . A path with no intermediate vertices is either the trivial path of length 0 (when $i = j$) or a single edge (i, j) (when $i \neq j$). Hence $d[i][j] = d^{(0)}[i][j]$.

Inductive hypothesis. Assume that after the $(k - 1)$ -th iteration, $d[i][j] = d^{(k-1)}[i][j]$ for all i, j .

Inductive step. We must show that after the k -th iteration, $d[i][j] = d^{(k)}[i][j]$ for all i, j .

Fix any pair (i, j) . A shortest path from i to j restricted to intermediates in $\{1, \dots, k\}$ either uses vertex k as an intermediate or it does not.

Case 1: The shortest path does not use k . Then all intermediates lie in $\{1, \dots, k - 1\}$, so its weight equals $d^{(k-1)}[i][j]$, which by the inductive hypothesis equals the current $d[i][j]$ before the k -th iteration.

Case 2: The shortest path uses k as an intermediate. Let p be such a path:

$$p = \langle i, \dots, k, \dots, j \rangle.$$

Decompose p into sub-paths $p_1 = \langle i, \dots, k \rangle$ and $p_2 = \langle k, \dots, j \rangle$. We claim that k does not appear as an intermediate in p_1 or p_2 .

- If k appeared as an intermediate in p_1 , then p_1 would contain a closed walk through k . Since G has no negative cycle, this walk has non-negative weight; removing it yields

a path from i to k that is at least as short, contradicting the assumption that p is a shortest path of minimum length.

- The symmetric argument applies to p_2 .

Hence all intermediates of p_1 and p_2 lie in $\{1, \dots, k-1\}$, giving

$$w(p_1) = d^{(k-1)}[i][k], \quad w(p_2) = d^{(k-1)}[k][j].$$

By the inductive hypothesis these equal $d[i][k]$ and $d[k][j]$ at the start of iteration k . Note also that the update rule reads $d[i][k]$ and $d[k][j]$ *before* modifying them (since the k -th row and column are unchanged by the update: $d[k][k] = 0$ ensures $d[k][j]$ cannot decrease through k itself, and similarly for $d[i][k]$). Therefore:

$$w(p) = w(p_1) + w(p_2) = d[i][k] + d[k][j].$$

Combining both cases:

$$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j]) = \min(d[i][j], d[i][k] + d[k][j]),$$

which is exactly the value assigned to $d[i][j]$ in the k -th iteration. The invariant is maintained. $\square$

Proof of Theorem. By the Lemma, after all n iterations, $d[i][j] = d^{(n)}[i][j]$ for every (i, j) . Since $d^{(n)}[i][j]$ is the weight of the shortest path from i to j allowing all n vertices as intermediates, it equals $\delta(i, j)$. Hence, when the algorithm terminates, $d[i][j] = \delta(i, j)$ for all $i, j \in V$. $\square$

Negative-Cycle Detection

Corollary 1. *After the algorithm terminates, G contains a negative cycle if and only if $d[v][v] < 0$ for some $v \in V$.*

Proof. ($\Rightarrow$) Suppose G contains a negative cycle $C = \langle v, v_1, \dots, v_m, v \rangle$ reachable from (and to) v . The algorithm considers all closed walks through v , so $d[v][v]$ is bounded above by $w(C) < 0$.

($\Leftarrow$) Suppose $d[v][v] < 0$ for some v . The value $d[v][v]$ records the weight of some walk from v back to v . A walk of negative weight implies a negative cycle along it (otherwise every cycle on the walk is non-negative, and removing cycles cannot decrease the weight below the direct path weight of $0 = \delta(v, v)$ in a cycle-free graph). $\square$

Remark 1. The correctness argument above assumes no negative cycle precisely because we used it to conclude that k does not repeat as an intermediate in p_1 or p_2 . If a negative cycle existed, a path could “wind around” it arbitrarily, making $\delta(i, j) = -\infty$ for some pairs, and the finite matrix d would not represent true shortest-path distances.

Comparison with Other Algorithms

Algorithm	Graph type	Complexity	Scope
Dijkstra (binary heap)	Non-negative weights	$O((V + E) \log V)$	Single source
Bellman–Ford	Any weights, detects neg. cycles	$O(V \cdot E)$	Single source
Floyd–Warshall	Any weights (no neg. cycles)	$O(V ^3)$	All pairs

Running Bellman–Ford from every source gives all-pairs distances in $O(|V|^2 \cdot |E|)$, which exceeds Floyd–Warshall’s $O(|V|^3)$ on dense graphs where $|E| = \Theta(|V|^2)$.

Complexity

The algorithm consists of three nested loops each running n times, with $O(1)$ work per innermost iteration:

$$T(n) = O(n^3), \quad n = |V|.$$

Space usage is $O(n^2)$ for the distance matrix d (the predecessor matrix for path reconstruction adds another $O(n^2)$ but does not affect the time bound).